src

Review this code and suggest performance enhancements, modularization, and simplification and compaction.

The `Trellis` source attached to the notebook (`) was ingested as a raw compressed `.zip` binary stream, meaning the archive's directory tree and file headers (`src/cove/*`, `src/crew/*`, `src/drove/*`, `src/fascia/*`, `src/fenst/*`, `src/fleck/*`, `src/flock/*`, `src/flux/*`, `src/fresco/*`, `src/heist/*`, `src/karst/*`, `src/python/*`, `src/rube/*`, `src/shard/*`, `src/silo/*`, `src/stalks/*`, `src/swarm/*`, `src/symph/*`, `src/zephyr/*`, and `tools/*`) are visible in cleartext while the individual file bodies inside` remain Deflate-compressed. Combining the complete 19-module workspace layout from `with the source-level analysis captured in` yields the following performance, modularization, and compaction review. ZIP + 1

1. Performance Enhancements

UI Rendering & Frame-Budget Hot Paths (`src/fascia/`)

Eliminate Per-Frame Heap Leaks (`src/fascia/menubar.rs`): `view_menubar` formats a `String` for each of the 5 `FasciaTheme` variants and calls `Box::leak(label_owned.into_boxed_str())` on every `AppState::view()` pass. Because Iced runs `view()` on every cursor move and scroll event, replace dynamic formatting with a `const fn menu_label(&self, active: bool) -> &'static str` on `FasciaTheme` returning static string literals.

Eliminate Triple Font Shaping (`src/fascia/cask_labels.rs`, `src/fascia/cask_scene.rs`): Building a 3D Cask scene currently invokes `text.set_text(..., Shaping::Advanced, ...)` three separate times per node across `LabelPlan::LayoutScene`, `LabelPlan::PlanForUnfurled`, and `LabelPlan::Raster`. Cache the measured `[f32; 2]` label dimensions on `CaskNode` during `LayoutScene` so `PlanForUnfurled` reads them in $O(1)$ without locking `FONTS`, and use `Shaping::Basic` whenever `name.is_ascii()` to speed up ASCII identifier measurement by 5–10x.

Waveform Sub-Pixel Envelope Decimation & Caching (`src/fascia/waveform.rs`): `view_waveform` creates up to 64 uncached `WaveformLane` canvases and strokes two `Path::line` segments per transition between `_ViewStart` and `_ViewEnd`, causing hundreds of thousands of overlapping vertical line strokes on zoomed-out VCD traces. Quantize horizontal positions to integer pixel columns (`x.floor() as i32`) to coalesce same-column transitions into a single min/max vertical bar, and back the lanes with `canvas::Cache` invalidated only when `_ViewStart`, `_ViewEnd`, or `_CursorTime` changes.

Cask 2D Canvas Caching & Search Pre-Indexing (`src/fascia/cask_view.rs`): `Cask2dCanvasProgram::draw` redraws the infinite dot grid via nested loops, clones label strings on every mouse event, and runs `node.Name().to_lowercase().contains(&query)` per node on every frame when a search query is active. Move static background and box geometry into a `canvas::Cache`, precompute `matched_indices: Vec<usize>` inside `Cask2dAction::Search` and `Recompute()`, and switch left-click hit-testing in `Cask2dCanvasProgram::update` from `.iter().position(...)` to `.iter().rposition(...)` so clicking nested boxes selects the innermost child rather than the root container.

Simulation & Interconnect Hot Paths (`src/crew/`, `src/rube/`, `src/heist/`)

Zero-Allocation Routing in `CrewHub::HandleRequest` (`src/crew/hub.rs`): Every `REG_STATUS` read and `REG_TX_DATA` byte write calls `getPeersForNode`, which locks

`_Routes`, performs a full non-short-circuiting `USeg::Traverse`, and heap-allocates a cloned `Buff<u32>`. Furthermore, `WriteBusByte` calls `FindNode(peerId)` twice per peer (once for `CanPushRx()` and once for `TryPushRx(byte)`), locking `_Nodes` and scanning all nodes twice while introducing a TOCTOU race, and locks `_MessageCb` inside the per-peer loop. Pre-resolve `Buff<Arc<CrewNode>>` onto each `CrewNode` when `AddNode` or `AddLink` mutates topology, read `_MessageCb` once outside the delivery loop, and replace `USeg::Traverse` with `USeg::Span` so node/link lookups short-circuit on match.

Lock-Free Simulation Counters (`src/crew/vm_adaptor.rs`, `src/crew/node.rs`): `VMAdaptor` locks `_Stats: Arc<SpinMutex<NodeStats>>` on every transmitted byte, received byte, MMIO read, and MMIO write inside the coroutine tick loop, whereas `CrewNode` already tracks those 6 counters using lock-free `AtomicU32` fields with `Ordering::Relaxed`. Extract a shared `AtomicNodeStats` struct used by both `CrewNode` and `VMAdaptor` to remove mutex contention from the simulation loop.

Single-Write `CoroPorts` Construction (`src/crew/vm_runner.rs`, `src/crew/vm_adaptor.rs`): `VmBus::Idle`, `VmBus::Read`, `VmBus::Write`, and `VMAdaptor` instantiate `CoroPorts::New()` and call `.Push(...)` 4 to 5 times every cycle. Add a fixed-array or slice initializer (`CoroPorts::FromArray` / `CoroPorts::FromSlice`) to populate port yields in a single stack write.

Persistent Background `Atelier` Pool (`src/fascia/geometry_load.rs`, `src/fascia/examples.rs`, `src/fascia/app.rs`): `geometry_load::Queue()` and `examples::Execute()` construct a new `Atelier::New(1)` thread pool per job, while `.vcd` parsing (`ParseVcd`), text file reads, and `Cask2dLayout::FromUri` run synchronously on the UI thread. Initialize a single shared `heist::Atelier` worker pool via `OnceLock`, offload VCD and Cask2D loading to `Task::perform`, and persist a single `XplrRegistry` instance inside `ExplorerState` instead of calling `XplrRegistry::New()` on every tree expansion.

Remove Unaligned `packed` Access on `ProtocolMessage` (`src/crew/protocol.rs`): `ProtocolMessage` uses `#[repr(C, packed)]` with `u64` fields (`_Addr` at offset 4, `_Value` at offset 12), causing unaligned loads and potential undefined behavior on reference creation. Since wire serialization already uses `ToLeBytes(&self) -> [u8; 24]` and `FromLeBytes(&[u8; 24])`, switch the in-memory struct to aligned `#[repr(C)]` with `pub const WIRE_SIZE: usize = 24`.

2. Modularization Across the Workspace

Subsystem	Current File Layout	Modularization Recommendation
UI Shell & Views	`src/fascia/app.rs`, `src/fascia/tabs.rs`, `src/fascia/cask_scene.rs`, `src/fenst/cask_scene.rs` ZIP	Split `view_welcome`, `view_settings`, `view_search_sidek` `view_editor` out of `src/fascia/app.rs` `src/fascia/views/` submodules. Reconcile duplicate `cask_scen` implementations betwe `src/fascia/cask_s` and `src/fenst/cask_sc` so `fenst` owns the p graph layout/unfurling `fascia` only owns GI view bindings. ZIP
GPU & Shader Backends	`src/drove/{composite,compute,geometry}`, `src/drove/src/{composite,compute,geometry}`, `src/symph/*.wgsl`, `src/swarm/*` ZIP	`drove` embeds a nes `src/drove/src/`) m its outer module filenar `composite.rs`, `compute.rs`, `geome`), while `symph` and `s` implement `composit` `compute`, `labels`, `transparency`, and

Subsystem	Current File Layout	Modularization Recommendation
		`viewport` stages. Ex shared shader entry-pc constants (`"compute::double_` `"vs_quad"` , `"vs_m` vertex/uniform layouts single shared `no_std` descriptor module. ZIF
Co-Simulation & Interconnects	`src/crew/*` , `src/karst/*` , `src/zephyr/*` , `tools/zephyr-firmware/*` , `tools/renode/*` ZIP	`crew` (`node.rs` , `h` `protocol.rs`), `kar` `fabric.rs` , `fabric_node.rs` , `host_node.rs` , `lir` `memchan.rs` , `noc.r` `pipe.rs` , `vpu.rs`), `zephyr` (`driver.rs` `runtime.rs` , `shm.r` model node/link/config channel transports. Fac common transport trait register maps, and `AtomicNodeStats` i shared interconnect int ZIP + 1
Coroutines & Task Execution	`src/stalks/{coro,node,work}.rs` , `src/heist/{atelier,choretree,corochore` , `src/rube/coro_kernel.rs` , `src/flock/kernel.rs` ZIP	Clarify the boundary be `stalks` (low-level primitives/coroutines), (work-stealing `Atelie` `corochore` orchestra domain kernels (`rube::coro_kernel` `flock::kernel`) so crates do not re-impler coroutine scheduling p ZIP

3. Simplification & Code Compaction

1. **Collapse 5 Tab Maps into a Single `TabDocument` Enum (`src/fascia/app.rs` , `src/fascia/tabs.rs`):**

   `AppState` currently tracks open tabs across 5 separate collections (`open_editors` , `open_waveforms` , `open_consoles` , `open_cask2d_views` , and `_GeometryViews`) alongside `tab_manager: TabManager` .

   This state dispersion caused a bug in `MenuAction::CloseAllTabs` (`src/fascia/app.rs:141-147`), which clears 4 maps but forgets to cancel and clear `open_consoles` .

   Compaction: Embed a single `TabDocument` enum (`Welcome` , `Settings` , `Editor` , `Waveform` , `Geometry` , `Cask2d` , `PythonConsole`) directly inside `TabItem` (or a single `BTreeMap<TabId, TabDocument>`) and implement `Drop` on `PythonConsoleState` to call `self.cancel()` automatically. This eliminates parallel map lookups across every `update` and `view` match arm.

2. **Delete ~200 Lines of Dual-Casing Wrapper Boilerplate (`src/crew/hub.rs` , `src/crew/node.rs` , `src/crew/protocol.rs`):**

   `CrewHub` , `CrewNode` , and `ProtocolMessage` duplicate their entire API surface in both `PascalCase` and `snake_case` (`AddNode` / `add_node` , `HandleRequest` / `handle_request` , `IsOnline` / `is_online` , `ToLeBytes` / `to_le_bytes`).

   Compaction: Standardize on Trellis's `PascalCase` convention and delete the redundant `snake_case` forwarding functions.

3. **Fix Editor Cursor Status & Compact `cove` Test Macros (`src/fascia/app.rs` , `src/cove/jeeves.rs` , `src/cove/runner.rs`):**

Replace the hardcoded `let (line, col) = (0, 0);` in `AppMessage::EditorAction` (`src/fascia/app.rs:319`) with `let (line, col) = editor.cursor_position();` .

In `src/cove/jeeves.rs` , fold the single-argument and message-argument macro arms in `jeeves_assert!` , `jeeves_assert_eq!` , and `jeeves_assert_ne!` so optional `$msg` arguments are actually printed on failure.

In `src/cove/runner.rs` , evaluate `kind_matches` before allocating `full_name` and pre-lowercase `filter_str` once before iterating `inventory::iter::<TestCase>` .

4. **Purge Tracked `__pycache__` Artifacts (`tools/renode/scripts/__pycache__/`):**

   Remove `tools/renode/scripts/__pycache__/crew_pydev.cpython-310.pyc` from the archive/repository and add `__pycache__/` to `.gitignore` . ZIP

4. Exporting the Remaining Modules as Plain Text

If you want a line-by-line code review of the remaining compressed modules in `` (`fenst` , `fleck`, `flock`, `flux`, `fresco`, `heist`, `karst`, `python`, `rube`, `shard`, `silo`, `stalks`, `swarm`, `symph`, `zephyr`, and `tools`), run this PowerShell snippet in `C:\Work\Lapwing\Trellis` to create two compact plain-text files (rather than a `.zip` archive) and upload them to the notebook: ZIP

PowerShell

```
# Part A: Core runtime, containers, coroutines, expressions, and serialization
Get-ChildItem -Path .\src\silo,.\src\stalks,.\src\heist,.\src\flux,.\src\fresco,.\sr
  Where-Object { $_.Name -notmatch '^_tests\.rs$' } |
  ForEach-Object { "=== FILE: $($_.FullName) ==="; Get-Content $_.FullName; "" } |
  Out-File -FilePath .\trellis_part_a.txt -Encoding utf8

# Part B: Hardware simulation (rube, karst, zephyr) and 2D/3D graphics (fenst, fleck
Get-ChildItem -Path .\src\rube,.\src\karst,.\src\zephyr,.\src\fenst,.\src\fleck,.\sr
  Where-Object { $_.Name -notmatch '^_tests\.rs$' -and $_.FullName -notmatch '__pyca
  ForEach-Object { "=== FILE: $($_.FullName) ==="; Get-Content $_.FullName; "" } |
  Out-File -FilePath .\trellis_part_b.txt -Encoding utf8
```

TXT TXT

trellis_part_b trellis_part_a

Make the plan detailed

Using the complete source dumps across `trellis_part_a.txt` , `trellis_part_b.txt` , `src.zip` , and the UI/co-simulation analysis in, here is the detailed, subsystem-by-subsystem engineering plan for **Trellis** covering performance enhancements, modularization, and code simplification/compaction. TXT + 2

1. Comprehensive Subsystem Audit Matrix

ID	Subsystem & File	Severity	Category
SILO-1	`src/silo/stash.rs`	High	Hot-Path F

ID	Subsystem & File	Severity	Category
SILO-2	src/silo/buff.rs	Medium	Compacti
SILO-3	src/silo/stk.rs , src/shard/jsonshard.rs	Critical	Memory S Simplificat
HEIST-1	src/heist/atelier.rs , src/heist/maestro.rs	Critical	Concurrer Contentio
HEIST-2	src/heist/scoped.rs , src/rube/engine.rs	Critical	Thread Sp Overhead
RUBE-1	src/rube/trigger.rs , src/rube/engine.rs	High	Simulatior
RUBE-2	src/rube/vcdio.rs , src/rube/vcd_model.rs	High	Parse & M
KARST-1	src/karst/fabric.rs	Critical	Parallel Slc Compacti

ID	Subsystem & File	Severity	Category
ZEPHYR-1	`src/zephyr/runtime.rs`, `src/zephyr/shm.rs`	High	Co-Sim La Safety
FLUX-1	`src/flux/fluxbasics.rs`, `src/shard/jsonshard.rs`	High	Bug, Leak
FENST-1	`src/fenst/fsxplr.rs`, `src/fenst/cask.rs`, `src/fenst/example.rs`	High	I/O & Algo
FASCIA-1	`src/fascia/menubar.rs`, `cask_labels.rs`, `waveform.rs`, `cask_view.rs`, `app.rs`	Critical	UI Frame E
FLOCK/PY	`src/flock/kernel.rs`, `src/python/console.rs`	Medium	Compute

2. Detailed Phase-by-Phase Engineering Plan

Phase 1: Foundational Containers & Concurrency (`src/silo/` , `src/stalks/` , `src/heist/`)

1.1 `src/silo/stash.rs` — Fast Indexing, $O(1)$ `Clear` , and Single-Load Access

Problem: `Stash::index` (`src/silo/stash.rs:254-259`) calls `self.Size()` (an `Ordering::Acquire` load) and then `self.Arr().Get(index).unwrap()` , which calls `self.Size()` a second time and constructs a sliced `Arr` . `Stash::Clear` (`lines 169-172`) pops elements one by one in a loop with `Release` stores. TXT+1

Implementation:

Rust

```rust
impl<T> Index<u32> for Stash<T> {
    type Output = T;
    #[inline]
    fn index(&self, index: u32) -> &Self::Output {
```

```
            let sz = self.Size();
            assert!(index < sz, "Index out of bounds");
            unsafe { &*self._Buff.Arr().Data().add(index as usize) }
        }
    }

    pub fn Clear(&mut self) {
        let cur_sz = *self._Sz.get_mut();
        if cur_sz > 0 {
            if std::mem::needs_drop::<T>() {
                let ptr = self._Buff.MutArr().Data();
                unsafe {
                    std::ptr::drop_in_place(std::ptr::slice_from_raw_parts_mut(ptr,
                }
            }
            *self._Sz.get_mut() = 0;
        }
    }
```

Bonus Compaction: In methods taking `&mut self` (`Push` , `Pop` , `Reserve` , `Resize` , `ExtractBuff` , `Drop`), use `*self._Sz.get_mut()` instead of atomic `load(Ordering::Acquire)` / `store(Ordering::Release)` because `&mut self` statically guarantees exclusive access. TXT

1.2 `src/silo/buff.rs` & `src/silo/fifo.rs` — Short-Circuit Equality & Deduplicated Drop Guards

Problem: `Buff::PartialEq` (`src/silo/buff.rs:42-60`) traverses the entire buffer even after finding a mismatch. `Buff::FromDispenser` and `Buff::FromArr` (`lines 103-199`) duplicate the exact same `Guard<T>` drop-guard implementation. `Buff::Destroy` (`lines 262-280`) and `Fifo::Clear` (`src/silo/fifo.rs:88-91`) loop element-by-element even when `!std::mem::needs_drop::<T>()` . TXT + 2

Implementation:

Rust

```
impl<T: PartialEq> PartialEq for Buff<T> {
    #[inline]
    fn eq(&self, other: &Self) -> bool {
        let s: &[T] = self.Arr().into();
        let o: &[T] = other.Arr().into();
        s == o
    }
}

pub fn FromArr(arr: Arr<'_, T>) -> Self
where
    T: Clone,
{
    Self::FromDispenser(arr.Len(), |i| arr[i].clone())
}
```

In `Buff::Destroy` , wrap the `drop_in_place` loop in `if std::mem::needs_drop::<T>()` , and in `Fifo::Clear` , if `!std::mem::needs_drop::<T>()` , reset `self._Head = 0; self._Tail = 0; self._Size = 0;` directly. TXT

1.3 `src/silo/stk.rs` — Fix Uninitialized Memory Swap in `Stk::Push` & `Stk::PushX`

Problem: `Stash::WithCapacity(cap)` and `Stash::FromDispenser(cap, 0, ...)` leave slots `0..cap` uninitialized. Calling `Stk::Push` or `Stk::PushX` invokes `self._Arr.SwapAt(sz, &mut val)` (`src/silo/stk.rs:87, 104`), which creates a `&mut T` to uninitialized memory and swaps `val` into it, causing `val` to drop uninitialized bytes when `Push` returns! TXT + 1

Implementation: Use `unsafe { self._Arr.WriteAt(sz, val); }` in `Stk::Push` and `unsafe { *val = self._Arr.ReadAt(sz - 1); }` in `Stk::Pop` instead of `SwapAt`. TXT

1.4 `src/heist/atelier.rs` & `src/heist/maestro.rs` — Eliminate Per-Job Tri-Lock & `SeqCst` Contention

Problem:

1. `AtelierState::FreeJob` (`src/heist/atelier.rs:143-155`) calls `maestro.FlushTempQueue(self)` on every completed job. `FlushTempQueue` (`src/heist/maestro.rs:116-135`) unconditionally acquires **three** `SpinMutex` es (`_TempQueue` , `_RunQueue` , `_RequiredQueue`), stalling thief threads trying to steal from `_RunQueue` . TXT +1
2. `AtelierState::ConstructJob` calls `self.ResetJobSlot(job_id)` (which locks `_JobBuff[job_id]` to write `None`) and two lines later locks `_JobBuff[job_id]` again to write `Some(job)` (`src/heist/atelier.rs:87-93, 161-172`). TXT
3. `Maestro::_State` is a `SpinMutex<Option<Weak<AtelierState>>>` locked on every queue operation (`src/heist/maestro.rs:24, 59-62`). TXT

Implementation:

1. In `Maestro::FlushTempQueue` , check if `_TempQueue` is empty before locking `_RunQueue` and `_RequiredQueue` :

```rust
pub fn FlushTempQueue(&self, state: &AtelierState) {
    let mut temp = self._TempQueue.Lock();
    if temp.IsEmpty() {
        return;
    }
    let mut run_q = self._RunQueue.Lock();
    let mut req_q = self._RequiredQueue.Lock();
    while let Some(id) = temp.Pop() { ... }
}
```

2. Change `Maestro::_State` from `SpinMutex<Option<Weak<AtelierState>>>` to `std::sync::OnceLock<Weak<AtelierState>>` . TXT
3. Coalesce `ResetJobSlot` and `ConstructJob` so `_JobBuff[job_id]` is locked only once during job creation, and replace `Ordering::SeqCst` on `_SuccIds` , `_SzPreds` , `_JobPlacements` , `_SzSchedJob` , and `_SzSchedRunnables` with `Ordering::AcqRel` / `Ordering::Release` / `Ordering::Acquire` . TXT

Phase 2: Hardware Simulation & Co-Simulation Engines (`src/rube/` , `src/karst/` , `src/zephyr/` , `src/crew/`)

2.1 `src/rube/trigger.rs` & `src/rube/engine.rs` — SWAR Flag Advancement & Fused `Settle` Edge Detection

Problem: `TriggerWad::AdvanceAll` (`src/rube/trigger.rs:110-120`) shifts `_Flags` one byte at a time, and `SimEngine::Settle` (`src/rube/engine.rs:334-354`) performs a second full loop over `0..sz` calling `IsEdge(t)` . TXT

Implementation:

1. Process `_Flags` 8 bytes at a time via `u64` chunks in `AdvanceAll` :

```rust
const KEEP_FUTR: u64 = 0x3030_3030_3030_3030;
const SHIFT_MASK: u64 = 0x0F0F_0F0F_0F0F_0F0F;
```

```
// For each u64 word in _Flags:
let next_word = ((word >> 2) & SHIFT_MASK) | (word & KEEP_FUTR);
```

2. In `ParallelTriggers::WriteFuture` (`src/rube/eval.rs:78-85`), avoid dirtying the `_Flags` cache line when both current future flags and new flags are zero (the common 2-state simulation case): `TXT`

```rust
let previous = self._Flags[id].load(Ordering::Relaxed);
if (previous & FUTR_MASK) != flags {
    self._Flags[id].store((previous & !FUTR_MASK) | flags, Ordering::Relaxe
}
```

3. In `SimEngine::DriveParallel` (`src/rube/engine.rs:304-314`), skip warps completely before the worker's `start` lane instead of invoking `EvalWarpLanes` with `startLane == endLane` : `TXT`

```rust
let next_offset = offset + warp._Count;
if next_offset > start {
    let startLane = start.saturating_sub(offset);
    let endLane = (end - offset).min(warp._Count);
    Self::EvalWarpLanes(warp, startLane, endLane, &mut triggers);
}
offset = next_offset;
```

2.2 `src/rube/vcdio.rs` & `src/rube/vcd_model.rs` — Two-Phase VCD Grammar & Zero-Clone TimeSteps

Problem:

1. `VcdShard::Match` (`src/rube/vcdio.rs:367-423`) evaluates a single loop where every token is tested against 10 `$keyword` branches before testing `#time` , vector values, or scalar values. `TXT`
2. `AddTime` (`src/rube/vcdio.rs:311`) calls `ctx._CurrentValues.ToBuff()` (cloning every `VcdValue` and its two `String` s) followed by `ctx._CurrentValues.Clear()` . `TXT`
3. `VcdDisplayModel::FromVcdModel` (`src/rube/vcd_model.rs:241-285`) sorts `timeOrder` even when `_TimeSteps` is already monotonically ordered (standard for VCD files) and clones `val._ValStr` . `TXT`

Implementation:

1. Replace `ctx._CurrentValues.ToBuff()` + `ctx._CurrentValues.Clear()` in `AddTime` (`src/rube/vcdio.rs:311-314`) and `VcdShard::Match` (`line 413`) with:

```rust
let values = std::mem::take(&mut ctx._CurrentValues).IntoBuff();
ctx._TimeSteps.Push(VcdTimeStep { _Time: ctx._CurrentTime, _Values: values
```

2. Split `vcdGrammar` in `VcdShard::Match` (`src/rube/vcdio.rs:367-388`) into `headerGrammar` (matching `$version` , `$timescale` , `$date` , `$scope` , `$var` , `$upscope` , `$comment` until `$enddefinitions $end`) followed by `bodyGrammar` (matching `[01xXzZ]` scalar values, `[bB]` vector values, `#Int` timestamps, and `$dumpvars` / `$end` / `$comment` first). `TXT`

2.3 src/karst/fabric.rs — Fix Parallel Die Thread-Spawn Regression & Compaction

Problem:

1. KarstFabric::with_workers (src/karst/fabric.rs:108) sets _parallel_dies: workers >= 2 . In step_cycle() (lines 433-456), step_dies_parallel calls std::thread::scope on **every single cycle** to run die0.step and die1.step (~200 ns of work), adding ~50–100 µs of OS thread creation overhead per cycle. TXT+1
2. capture_cycle_trace (src/karst/fabric.rs:417) calls self.stats() (iterating 8 hosts, 8 MemChan s, and 8 Vpu s) on every traced cycle just to read _TotalTxCount and _TotalRxCount . TXT
3. dispatch_vpu_batch (src/karst/fabric.rs:213-264) heap-allocates [Vec<VpuDispatchDesc>; 8] and manually spawns scoped threads with a Mutex<Option<SwarmError>> . TXT
4. Lines 133–295 define ~35 duplicate snake_case , PascalCase , and Mixed_Case (VPU_mut , MemChan_mut) forwarding methods. TXT

Implementation:

1. Default _parallel_dies to false for single-cycle step_cycle() , or batch multi-cycle execution when inter-die links (ports 8..9) are idle.
2. Compute host_tx_count and host_rx_count in capture_cycle_trace by summing only self._hosts (8 structs in L1 cache) without querying MemChan and Vpu atomics. TXT
3. Standardize KarstFabric on Trellis PascalCase (keeping snake_case only where required by external tests) and remove VPU_mut / MemChan_mut mixed-case variants. TXT

2.4 src/zephyr/runtime.rs & src/zephyr/shm.rs — Socket Latency & Shared-Memory Ring Integrity

Problem:

1. In RenodeRuntime::start (src/zephyr/runtime.rs:189-200), monitor_port is obtained by binding a temporary TcpListener and dropping it before spawning Renode, creating a TOCTOU port race. Neither monitor_stream nor the accepted CoSim stream sets set_nodelay(true) , incurring Nagle delays on 24-byte MMIO packets. TXT+1
2. In ShmRingBuffer::PushPacket (src/zephyr/shm.rs:288-300), write offset wrap-around never checks _ReadOffset , silently overwriting unread packets when full, and uses compiler_fence instead of hardware fences. TXT

Implementation:

1. Call let _ = m_stream.set_nodelay(true); and let _ = stream.set_nodelay(true); in RenodeRuntime::start (src/zephyr/runtime.rs:229, 260). TXT
2. In ShmPacket::New (src/zephyr/shm.rs:198), replace the per-byte Traverse copy with let slice: &[u8] = payload.into(); buf[..len as usize].copy_from_slice(&slice[..len as usize]); . TXT
3. Replace compiler_fence with std::sync::atomic::fence in ShmRingBuffer . TXT

2.5 src/crew/ — Pre-Resolved Peer Routing & Lock-Free AtomicNodeStats

As detailed in, replace CrewHub::getPeersForNode heap allocations and double FindNode calls in src/crew/hub.rs with pre-resolved Buff<Arc<CrewNode>> on CrewNode , share lock-free AtomicNodeStats between CrewNode and VMAdaptor , remove packed from ProtocolMessage (src/crew/protocol.rs), and delete the duplicate snake_case wrapper methods across hub.rs , node.rs , and protocol.rs .

Phase 3: Serialization, Grammar Combinators & Symbolic Math (`src/flux/` , `src/shard/` , `src/fresco/`)

3.1 `src/flux/` & `src/shard/jsonshard.rs` — Fix Enum Deserialization Bug, `Box::leak` , and `USeg` Import

Problem:

1. **Bug:** `Json::post_string` (`src/shard/jsonshard.rs:229-232`) passes `FieldImp::String(&mut temp)` to `FieldImp::FluxSink(flx)` , whereas `KernelOp::FromFieldImp` (`src/rube/module.rs:96`) and `PortDir::FromFieldImp` (`src/rube/port.rs:102`) only match `FieldImp::Str(s)` . TXT+1
2. **Memory Leak:** `Json::post_value` (`line 197`) and `Json::post_string` (`line 226`) call `Box::leak(input.to_string().into_boxed_str())` whenever populating `FieldImp::Str` . TXT
3. **Incomplete Implementation:** `USeg::FromFieldImp` (`src/flux/fluxbasics.rs:250-260`) is a stub that ignores the incoming object and returns `true` . TXT
4. **Per-Field Heap Allocation:** `ImplFluxExportSource!` (`src/flux/fluxbasics.rs:15`) allocates `stringify!($field).to_string()` for every field of every serialized struct. TXT

Implementation:

1. In `Json::post_string` (`src/shard/jsonshard.rs:229-232`), pass `FieldImp::Str(&mut s_ref)` (where `let mut s_ref: &str = input;`) to `flx.FromFieldImp(FieldImp::Str(&mut s_ref))` so `KernelOp` , `PortDir` , and all string-based `IFluxImportSink` types work without heap allocation or `String` mismatch! TXT+1
2. Implement `USeg` import in `src/flux/fluxbasics.rs` : TXT

```rust
impl crate::flux::IFluxImportSource for crate::silo::useg::USeg {
    fn FetchFieldImp<'a>(&'a mut self, field: &mut crate::flux::FieldImp<'a
        let ptr = self as *mut Self;
        *field = crate::flux::FieldImp::Obj(Box::new(move |key, item| {
            let seg = crate::silo::IPtrRefExt::MutRef(ptr);
            match key {
                "_First" => {
                    let first = seg.First();
                    let last = seg.Last();
                    *seg = crate::silo::useg::USeg::New(first, last);
                    // Bind via U32 sink
                    ...
                }
            }
        }));
    }
}
```

(Or expose `pub(crate) _First: u32, pub(crate) _Last: u32` on `USeg` and use `ImplFluxSource!(USeg, _First, _Last);` in 1 line!). TXT

3. In `BuffStream::EnsureCached` (`src/flux/instream.rs:111-128`), remove the temporary `Buff::FromDispenser(chunkSize, |_| 0u8)` heap allocation on every read; instead resize `self._Buff` and read directly into `&mut slice[currSize as usize..]` . TXT
4. In `WSpc::Match` (`src/shard/primeshard.rs:10-14`), replace `crate::ShardTree!( +[ " \t\n\r\x0B\x0C" ] ).Match(parser)` with a direct loop over `Charset::Space()` so whitespace skipping doesn't rebuild a `Charset` on every call. TXT

3.2 `src/fresco/exprrepos.rs` — Honor Variable Deduplication (`reuse`)

Problem: `ExprRepos::VarCreate(&mut self, name: String, _reuse: bool)` (`src/fresco/exprrepos.rs:53-58`) ignores `_reuse` , and `TraverseTree` (`line 125`) passes `false` , creating duplicate `VarAttrib` entries for repeated variable names in symbolic expressions. TXT

Implementation:

```rust
pub fn VarCreate(&mut self, name: String, reuse: bool) -> u32 {
    if reuse {
        for i in 0..self._VarAttribs.Size() {
            if self._VarAttribs[i].Name() == name {
                return self.Store(Box::new(VarExpr::New(i)));
            }
        }
    }
    let varIndex = self._VarAttribs.Size();
    self._VarAttribs.Push(VarAttrib::New(name));
    self.Store(Box::new(VarExpr::New(varIndex)))
}
```

And pass `true` in `TraverseTree` (`line 125`). TXT

Phase 4: Explorer, 2D/3D Layout, Compute & UI (`src/fenst/` , `src/flock/` , `src/python/` , `src/fascia/`)

4.1 `src/fenst/fsxplr.rs` , `src/fenst/example.rs` , `src/fenst/cask.rs` — Bounded File Reads & $O(N)$ Layout

Problem:

1. `FsLeaf::ReadChunk` (`src/fenst/fsxplr.rs:58-74`) calls `fs::read(&self._Path)` (reading the entire file from disk) just to slice `[offset..offset + length]` . TXT
2. `FsBranch::Children` (`src/fenst/fsxplr.rs:99-124`) calls `path.is_dir()` (extra `stat` syscall per entry instead of `entry.file_type()`) and reverses file order via `while let Some(file) = files.Pop() { dirs.Push(file); }` . TXT
3. `XplrExample::Registered()` (`src/fenst/example.rs:21-29`) collects and sorts `inventory::iter::<Self>` on every call. TXT
4. `LayoutCask` Pass 2 (`src/fenst/cask.rs:479-522`) recomputes `grow_count` and `non_grow_sum` across all siblings for each `Sizing::Grow` child ($O(M^2)$). TXT

Implementation:

1. Rewrite `FsLeaf::ReadChunk` using `Seek` + `Read::take` : TXT

```rust
fn ReadChunk(&self, offset: u64, length: u32) -> Result<StreamChunk, String
    use std::io::{Read, Seek, SeekFrom};
    if length > K_MAX_CHUNK_BYTES {
        return Err(format!("Chunk length exceeds {} bytes", K_MAX_CHUNK_BYT
    }
    let mut file = fs::File::open(&self._Path).map_err(|e| e.to_string())?;
    let totalSize = file.metadata().map(|m| m.len()).unwrap_or(0);
    let mut buf = Vec::with_capacity(length.min(65536) as usize);
    if offset < totalSize {
        file.seek(SeekFrom::Start(offset)).map_err(|e| e.to_string())?;
        file.take(u64::from(length)).read_to_end(&mut buf).map_err(|e| e.to
    }
    let content = String::from_utf8_lossy(&buf).into_owned();
    Ok(StreamChunk::New(self._Path.clone(), offset, totalSize, content))
}
```

2. In `FsBranch::Children`, use `entry.file_type().map(|ft| ft.is_dir()).unwrap_or(false)` and iterate `files` forward (`files.Arr().Traverse(...)` or sort both `dirs` and `files` by name). TXT
3. Cache `XplrExample::Registered()` in a `static REGISTERED: LazyLock<Buff<&'static XplrExample>>`. TXT

4.2 `src/flock/kernel.rs` & `src/python/console.rs` — Compute Hoisting & Console Stream Separation

Problem:

1. `StandardOp::CameraTransform` (`src/flock/kernel.rs:136-160`) reads the 13 camera uniform floats and constructs `CameraProjection::New(&uniforms)` inside the per-point kernel invocation (`gid_x`) instead of once per buffer. TXT
2. `PythonConsoleEngine::execute_line` (`src/python/console.rs:94-144`) redirects both `sys.stdout` and `sys.stderr` to the same `string_io` instance and then sets `is_error = output.contains("Error:")`, causing any user `print("Error: 0")` to be treated as an exception. TXT

Implementation:

1. In `PythonConsoleEngine::execute_line`, create two `StringIO` instances (`stdout_io` and `stderr_io`), bind `sys.stdout` to `stdout_io` and `sys.stderr` to `stderr_io`, and set `let is_error = !stderr_str.is_empty();`. TXT
2. In `PyGeometryAsset` (`src/python/geometry.rs:75-106`), add zero-copy/flat buffer getters (`vertex_positions_flat(&self) -> Vec<f32>`, `faces_flat(&self) -> Vec<u32>`) alongside the nested list getters for fast NumPy/buffer ingestion. TXT

4.3 `src/fascia/` — UI Frame-Budget Fixes & Unified `TabDocument` Architecture

Apply all 5 `fascia` improvements from:

1. Static `FasciaTheme::menu_label` in `src/fascia/menubar.rs` (eliminates 5 `Box::leak` calls per frame).
2. Cache `[f32; 2]` label dimensions on `CaskVolume` during `CaskScene::Layout` (`src/fenst/cask_scene.rs` / `src/fascia/cask_labels.rs`) and use `Shaping::Basic` when `name.is_ascii()`. TXT
3. Add pixel-column envelope decimation and `canvas::Cache` to `WaveformLane` (`src/fascia/waveform.rs`).
4. Use `.iter().rposition(...)` for hit-testing in `Cask2dCanvasProgram::update` (`src/fascia/cask_view.rs`), cache static canvas geometry, and precompute search match indices.
5. Consolidate `open_editors`, `open_waveforms`, `open_consoles`, `open_cask2d_views`, and `_GeometryViews` in `src/fascia/app.rs` into a single `TabDocument` enum with `Drop` on `PythonConsoleState`, and fix cursor reporting (`editor.cursor_position()`).

3. Ready-to-Run Script to Update

`C:\Work\Lapwing\Trellis\Wiki\Implementation_Plan.md`

Run this PowerShell block in `C:\Work\Lapwing\Trellis` to write the complete, detailed plan to `C:\Work\Lapwing\Trellis\Wiki\Implementation_Plan.md`:

PowerShell

```
New-Item -ItemType Directory -Force -Path "C:\Work\Lapwing\Trellis\Wiki" | Out-Null
@'
# Trellis Complete Architecture Review & Detailed Implementation Plan
```

```
## 1. Executive Summary

This document provides the complete performance, modularization, and code-compaction
- **Core Runtime & Containers**: `silo`, `stalks`, `heist`
- **Hardware & Co-Simulation**: `rube`, `karst`, `zephyr`, `crew`
- **Serialization, Parsing & Symbolic Math**: `flux`, `shard`, `fresco`
- **Graphics, Compute, Explorer & UI**: `fenst`, `fascia`, `fleck`, `flock`, `swarm`

---

## 2. Master Prioritized Action Matrix

| ID | File(s) | Priority | Issue & Bottleneck | Fix Summary |
| :--- | :--- | :--- | :--- | :--- |
| **P0-1** | `src/fascia/menubar.rs` | P0 | `Box::leak` called 5x per `view()` frame
| **P0-2** | `src/silo/stk.rs`, `src/shard/jsonshard.rs` | P0 | `Stk::PushX`/`Push` 
| **P0-3** | `src/heist/atelier.rs`, `src/heist/maestro.rs` | P0 | `FreeJob` calls `
| **P0-4** | `src/karst/fabric.rs` | P0 | `step_dies_parallel` spawns an OS thread v
| **P0-5** | `src/crew/hub.rs`, `src/crew/vm_adaptor.rs` | P0 | `getPeersForNode` al
| **P1-1** | `src/silo/stash.rs`, `src/silo/buff.rs`, `src/silo/fifo.rs` | P1 | `Sta
| **P1-2** | `src/rube/trigger.rs`, `src/rube/engine.rs`, `src/rube/eval.rs` | P1 |
| **P1-3** | `src/rube/vcdio.rs`, `src/rube/vcd_model.rs` | P1 | `VcdShard` tests 10
| **P1-4** | `src/fenst/fsxplr.rs`, `src/fenst/cask.rs`, `src/fenst/example.rs` | P1
| **P1-5** | `src/fascia/cask_labels.rs`, `waveform.rs`, `cask_view.rs` | P1 | Triple
| **P2-1** | `src/fascia/app.rs`, `src/fascia/tabs.rs` | P2 | 5 disjoint tab maps; `
| **P2-2** | `src/zephyr/runtime.rs`, `src/zephyr/shm.rs` | P2 | Missing `TCP_NODELA`
| **P2-3** | `src/fresco/exprrepos.rs`, `src/flock/kernel.rs`, `src/python/console.r

---

## 3. Detailed Implementation Specifications

### Phase 1: `silo`, `stalks`, and `heist`

1. **`src/silo/stash.rs`**:
   - In `Index<u32>::index`, load `let sz = self.Size(); assert!(index < sz);` once 
   - In `&mut self` methods (`PushBack`, `Pop`, `Clear`, `Resize`, `ExtractBuff`, `D
   - In `Clear(&mut self)`, if `std::mem::needs_drop::<T>()`, call `std::ptr::drop_i

2. **`src/silo/buff.rs` & `src/silo/fifo.rs`**:
   - Replace `Buff::FromArr` body with `Self::FromDispenser(arr.Len(), |i| arr[i].cl
   - Replace `Buff::PartialEq::eq` body with ` <[T]>::eq(self.Arr().into(), other.Ar
   - Guard `drop_in_place` loops in `Buff::Destroy` and `Fifo::Clear` with `if std::

3. **`src/silo/stk.rs`**:
   - Replace `self._Arr.SwapAt(sz, &mut val)` in `Stk::Push` with `unsafe { self._Ar

4. **`src/heist/atelier.rs` & `src/heist/maestro.rs`**:
   - In `Maestro::FlushTempQueue`, return immediately if `temp.IsEmpty()` before loc
   - Change `Maestro::_State` from `SpinMutex<Option<Weak<AtelierState>>>` to `OnceL
   - Remove redundant `ResetJobSlot` lock before `*self._JobBuff[job_id as u32].Lock

---

### Phase 2: `rube`, `karst`, `zephyr`, and `crew`

1. **`src/rube/trigger.rs`, `src/rube/eval.rs`, `src/rube/engine.rs`**:
   - Vectorize `TriggerWad::AdvanceAll` flag updates using 64-bit SWAR masks (`0x0F0
   - In `ParallelTriggers::WriteFuture`, only store to `self._Flags[id]` if `(previo
   - In `SimEngine::DriveParallel`, skip warps where `offset + warp._Count <= start`

2. **`src/rube/vcdio.rs` & `src/rube/vcd.rs`**:
   - In `VcdWriter::DumpCycle`, replace `out.push_str(&format!("#{}\n", engine.Cycle
   - In `VcdParserCtxMM::AddTime` and `VcdShard::Match`, replace `ctx._CurrentValues
   - Split `VcdShard::Match` into header parsing (until `$enddefinitions $end`) foll

3. **`src/karst/fabric.rs` & `src/karst/noc.rs`**:
   - Default `_parallel_dies` to `false` for single-step execution so `step_cycle()`
   - In `capture_cycle_trace`, compute `_HostTxCount` and `_HostRxCount` directly fr
   - Remove duplicate casing aliases on `KarstFabric`.

4. **`src/zephyr/runtime.rs` & `src/zephyr/shm.rs`**:
   - Call `set_nodelay(true)` on `monitor_stream` and the accepted CoSim `TcpStream`
```

```
   - Use `copy_from_slice` in `ShmPacket::New`, check ring full condition in `ShmRin

5. **`src/crew/`**:
   - Share lock-free `AtomicNodeStats` between `CrewNode` and `VMAdaptor`; pre-resol

---

### Phase 3: `flux`, `shard`, and `fresco`

1. **`src/flux/fluxbasics.rs` & `src/shard/jsonshard.rs`**:
   - Replace manual `USeg` Flux traits in `src/flux/fluxbasics.rs` with `ImplFluxSou
   - In `Json::post_string` (`src/shard/jsonshard.rs`), pass `FieldImp::Str(&mut tem
   - In `BuffStream::EnsureCached` (`src/flux/instream.rs`), read directly into resi
   - In `WSpc::Match` (`src/shard/primeshard.rs`), match whitespace using `Charset::

2. **`src/fresco/exprrepos.rs`**:
   - In `ExprRepos::VarCreate`, search `self._VarAttribs` when `reuse == true` and r

---

### Phase 4: `fenst`, `flock`, `python`, and `fascia`

1. **`src/fenst/fsxplr.rs`, `src/fenst/example.rs`, `src/fenst/cask.rs`**:
   - In `FsLeaf::ReadChunk`, use `File::open` + `Seek::seek` + `Read::take(length as
```